PROJECT REPORT - CUSTOM SHELL

Name: Adyant Kar

Admission Number: 22JE0067

Group Number: 25

---

1. Introduction

This project involves developing simplified UNIX-like utilities in C, along with a custom-built shell to execute them. The shell acts as an interface between the user and the operating system, allowing users to run commands and interact with the system.

In this project, I was responsible for implementing the **custom shell (custom_shell)**, which serves as the core component. It accepts user input, processes commands, and executes both built-in and external utilities.

The project provides practical exposure to system-level programming concepts such as **process creation, command parsing, and execution using system calls**.

---

2. Objective

- To design and implement a custom UNIX-like shell
- To understand how commands are executed internally
- To use system calls like fork(), exec(), and wait()
- To build an interactive command-line interface

---

3. Features of Custom Shell

- Interactive command prompt
- Built-in commands:
    - cd – change directory
    - pwd – print working directory
    - help – display commands
    - exit / quit – terminate shell
- Execution of external utilities (custom_ls, custom_cat, etc.)

- Error handling for invalid commands
- Colored terminal output
- Automatic utility path resolution

---

4. Concepts Used

- Process creation (fork())
- Program execution (execvp())
- Process synchronization (waitpid())
- File system navigation (chdir(), getcwd())
- String parsing (strtok())
- Environment variables (getenv())

---

5. Code Explanation

The implementation of the custom shell is provided in .

◆ **Input Handling**

- The shell reads user input using fgets()
- Input is parsed into tokens using strtok()

◆ **Built-in Commands**

- cd → changes directory using chdir()
- pwd → prints current directory using getcwd()
- help → displays available commands
- exit → terminates shell

◆ **Command Execution**

- fork() creates a child process
- execvp() executes the command
- Parent process waits using waitpid()

◆ **Utility Resolution**

- Checks:

- Same directory
- Current working directory
- System PATH

---

6. Execution Steps

Compile:

make

Run Shell:

./custom_shell

---

7. Sample Output

Based on the implementation described in :

Welcome to Custom Shell! Type 'help' for commands.

```
Welcome to Custom Shell! Type 'help' for commands.
custom_shell:~$ help
╔═══════════════════════════════════════════════════════╗
║           Custom UNIX-like Shell                      ║
║         Supported Commands                            ║
╚═══════════════════════════════════════════════════════╝
Built-in commands:
  cd [dir]     - Change directory
  pwd          - Print working directory
  help         - Show this help message
  exit / quit - Exit the shell

Custom utilities:
  custom_ls   [dir]              - List directory contents
  custom_cat  <file> [file...]   - Display file contents
  custom_grep <pattern> <file>   - Search pattern in file
  custom_wc   <file> [file...]   - Word/line/char count
  custom_cp   <src> <dst>        - Copy file
  custom_mv   <src> <dst>        - Move / rename file
  custom_rm   [-r] <path>        - Remove file or directory
custom_shell:~$
```

---

8. Screenshots

- Shell running
- Executing commands
- Output of utilities

---

9. Conclusion

The custom shell implementation helped in understanding how command-line interfaces work internally. It provided hands-on experience with process management, system calls, and command execution in a UNIX-like environment.

This project strengthened my knowledge of **operating systems concepts and C programming**, especially in building real-world system utilities.